Find Missing Abstractions

Duplication of structure often means there's a missing abstraction, which in Ruby generally means you can move some code into a new class.

One symptom is a set of instance attributes that are always used together—especially if the same group of attributes is being passed to multiple methods. Use of a common set of variables often indicates that you want a new class with those friendly attributes as the instance attributes.

Another common symptom is a group of methods that all share a prefix or a suffix, such as `logger_init`, `logger_print`, and `logger_read`. Often this means you need a class corresponding to the common word.

In ActiveRecord, one side effect of discovering friendly attributes is the creation of value objects, which are immutable instances that represent parts of your data. For example, a `start_date` and `end_date` are often used together and could easily be combined into a `DateRange` class. Also, how often do you write something like this?

```
class User < ActiveRecord::Base
  def full_name
    "#{first_name} #{last_name}"
   end
   def sort_name
     "#{last_name}, #{first_name}"
   end
 end
```

You could try this:

```
class Name
  attr_reader :first_name, :last_name
  def initialize(first_name, last_name)
    @first_name, @last_name = first_name, last_name
  end
  def full_name
    "#{first_name} #{last_name}"
   end
   def sort_name
     "#{last_name}, #{first_name}"
   end
 end

 class User < ActiveRecord::Base
   delegate :full_name, :sort_name, to: :name
   def name
     Name.new(first_name, last_name)
   end
 end
```